COMMUNITY ENGINEERING GUIDE

PSP EBOOT:
Building for Official Firmware & Paranoid Cross-Validation

Understand the format. Protect the original.
Build, sign and verify a complete SampleApp.

THREE PRACTICAL LABORATORIES

Safe Signing Laboratory
PRX / NID Auditor
Reproducible SampleApp Laboratory

Target metadata: OFW 6.60 / 6.61 investigation
Offline validation and PPSSPP evidence supplied.
Physical stock OFW: NOT TESTED.

Contents

How to Use This Handbook 5

Claims have a scope . . . 5

Read or reproduce . . . 6

Chapter 1 — PSP Firmware Architecture 7

From a file on storage to application behavior . . . 7

OFW, CFW and launch context . . . 7

User modules and kernel interfaces . . . 8

PRX modules and service lifecycles . . . 8

A useful dependency policy . . . 8

Chapter 2 — PSP Hardware Generations 9

Physical specifications and runtime budgets . . . 9

RAM, embedded DRAM and VRAM are different fields . . . 9

Installation and audio consequences . . . 9

A model is a test condition . . . 10

Chapter 3 — Signing Tools Compared 11

“Sign” names several different transformations . . . 11

The selected legacy path . . . 11

SDK PrxEncrypter . . . 12

sign_np and PSPPack . . . 12

Choosing and preserving a tool . . . 12

Chapter 4 — PRX/NID Compatibility Analyzer 13

Inventory the binary that will be signed . . . 13

Classification with an explicit evidence boundary . . . 13

The corrected Kernel_Library example . . . 14

Investigating an unknown dependency . . . 14

Chapter 5 — Memory & Performance Engineering **15**

Build a budget from distinct quantities . . . 15

Heap, stack and allocation lifetime . . . 16

Threads, CPU and VFPU . . . 16

Graphics and stream-buffer arithmetic . . . 16

Chapter 6 — EBOOT Forensics and Safe Signing **17**

Read the PBP as a bounded container . . . 17

PARAM.SFO deserves its own parser . . . 17

Validate ELF and PRX before legacy code reads them . . . 18

The allowed relocation change . . . 18

Isolate the signer and protect the original . . . 19

Independent cryptographic cross-validation . . . 19

Extract components without losing provenance . . . 19

Chapter 7 — Reproducible Build Laboratory **20**

Preserve the environment as an input . . . 20

Windows 11, WSL2 and Linux . . . 20

Audit main.c and the Makefile together . . . 20

Build twice without sharing outputs . . . 22

Executed commands and measured artifacts . . . 22

Chapter 8 — Automated Test Laboratory **26**

Tests should challenge the claimed property . . . 26

Structural and metadata controls . . . 26

Cryptographic and continuity controls . . . 26

Process safety and archive controls . . . 27

PPSSPP integration . . . 27

Evidence ready for review . . . 27

Chapter 9 — Common PSP Development Mistakes **30**

Diagnose the stage, not the folklore . . . 30

Folder names and installation structure . . . 30
Metadata and stale generated files . . . 30
Unsupported imports and privilege confusion . . . 31
HOME callback, memory and apparent hangs . . . 31

Chapter 10 — Community Release Checklist **32**

Construct a new staging tree . . . 32
Create a fresh archive . . . 32
Verify the exact member set and content . . . 32
Write a README a reader can act on . . . 33
A full publication workflow . . . 33
Compatibility matrix discipline . . . 34

Chapter 11 — Common PSP Homebrew Myths & Misconceptions **35**

Signed EBOOT means guaranteed OFW launch . . . 35
PPSSPP PASS means PSP hardware PASS . . . 35
A 4 MiB signer buffer is the application's RAM limit . . . 35
Every sceKernel function requires Kernel Mode . . . 35
A matching SHA-256 proves the program is correct . . . 35
Successful PRX decryption proves Sony's loader will accept it . . . 35
CFW compatibility establishes stock OFW compatibility . . . 35
A successful command completes the release . . . 35

Primary Sources and Reference Identities **36**

How to Use This Handbook

This handbook follows one complete path from a small C application to a checked EBOOT and a publishable companion archive. The aim is a result that another developer can inspect and repeat. A signing program producing a file is one event in that path. The container must also be coherent, its metadata must match the application, its executable must fit the selected parser and signer, its imports need an evidence trail, and its release must contain the files described by its documentation.

The working example is SampleApp. It is deliberately small so that packaging, privilege, dependency and verification failures remain visible. It is not a demonstration of every PSP subsystem. A larger application should first reproduce this baseline, then add one dependency or behavior at a time. Every resulting build has a new identity and needs its own reports. The neutral example does not inherit results from an older SampleApp, another project, a similar executable, or an unsigned development build.

The document and companion are a pair. The PDF explains the decisions, formats, failure modes and procedures. The companion contains the complete laboratory source, tests, example, reports and reproducibility records listed by its manifest. Printed internal functions are explicitly excerpts when they are not complete files. Use the attached source for execution; do not reconstruct a tool by joining unrelated fragments from the PDF. The exact commands and current measured results appear in the laboratory chapters.

Claims have a scope

Use the following vocabulary throughout a report. Keep the name of the check beside its result; a bare green PASS hides too much information.

Status	Meaning in this handbook
PASS	A named check was executed and its stated criterion succeeded for the identified inputs and environment.
FAIL	A named check was executed and its criterion did not succeed. Preserve the failure and its evidence.
NOT TESTED	The operation or compatibility case has not been executed.
NOT MEASURED	A quantitative property, such as a duration or peak memory use, has not been measured.
UNKNOWN	Available evidence is insufficient to decide the property.
VERIFIED	In an import report, an exact mapping was found in the selected pinned public SDK user stubs. It means SDK mapping only.
UNSUPPORTED	A specific, documented conflict with this laboratory's chosen profile has been established. It is not a universal verdict about all PSP software.

A PASS in structural parsing is not a cryptographic PASS. A cryptographic PASS is not evidence that Sony's loader accepts the executable. An emulator observation is not a physical-device observation. These distinctions are practical: they tell the next investigator which boundary still needs evidence.

The word official describes stock Sony firmware when discussing OFW, and the PSPDEV project's own published environment when discussing its development image. PSPDEV and PSPSDK are community development projects, not Sony's proprietary SDK. Public SDK source is primary evidence for what that SDK declares. Sony manufacturer material is primary evidence for advertised hardware. Neither replaces testing the application on the specified firmware and model.

Read or reproduce

Read Chapters 1–3 to understand the platform and the tool choices. Chapters 4–6 explain what the laboratory examines in an executable and why. Chapters 7–8 provide the build and verification workflow. Chapters 9–10 turn diagnoses into an installation and community release. The final chapter collects myths that often cause evidence from one stage to be overstated at another.

Keep original PDFs, firmware references, user binaries and distribution archives unchanged. Perform experiments on newly created working copies. The laboratory does not require distributing Sony firmware binaries. A reader who only wants to inspect the release can verify its external hashes, open the companion manifest and read the reports without launching a signer or installing anything on a PSP.

EVIDENCE BOUNDARIES

Offline checks	Emulator	Physical device
Structure + crypto	Modeled behavior	Model + stock firmware

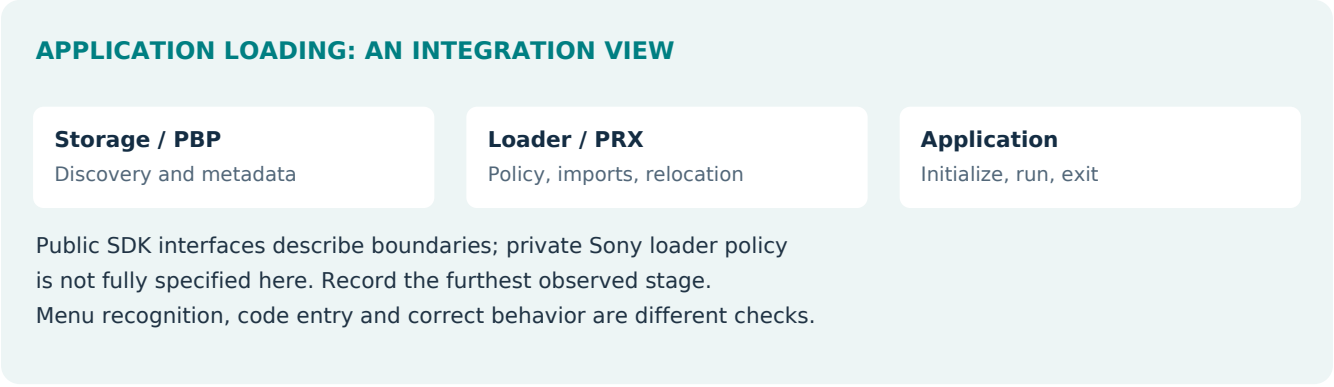
PASS stays attached to its own check and artifact hash.
A result at one boundary does not fill another boundary with PASS.
Physical stock OFW in this laboratory: NOT TESTED.

Chapter 1 — PSP Firmware Architecture

From a file on storage to application behavior

It helps to think of application loading as a sequence of boundaries. The storage layout lets the system find a candidate. The PBP wrapper supplies metadata and resources. DATA.PSP supplies the executable content for the chosen format. Executable metadata describes the module and its imports. Loader decisions, relocation and dependency resolution precede the application's own initialization. A running program must then manage memory, callbacks and subsystem errors correctly.

This is an integration view, not a complete specification of Sony's private boot implementation. The public sources cited here do not fully establish the internal boot chain, every cryptographic policy or every firmware memory map. The practical sequence still gives useful debugging stages. An XMB entry proves something different from reaching the first instruction of SampleApp, and reaching that instruction proves something different from a working exit callback.



Record the furthest observed boundary without guessing the cause of the next failure. If an entry is displayed but launch fails, preserve the exact message, executable hash, model and displayed firmware. If the first screen appears and initialization later fails, retain the failing API stage and numeric return value. A photograph of a title in the menu cannot establish that the executable ran.

OFW, CFW and launch context

OFW means stock Sony system software in this handbook. CFW describes a modified firmware environment that can change loading or execution behavior. A program working in a CFW environment does not establish stock OFW acceptance. Likewise, changing a launch route can change the context under investigation. Keep the firmware state and launch method explicit rather than describing every result simply as “works on PSP.”

The guide does not ask the reader to replace a preserved firmware installation to obtain a test result. A hardware record should identify the existing environment and describe how its stock or modified state was established. When that state cannot be determined, record UNKNOWN. A result with uncertain firmware identity cannot become a clean OFW compatibility claim by omission.

Compiler targeting is a separate concept. A compile-time firmware macro controls source and build choices. SFO metadata can declare a system version. The pinned sysmem header also distinguishes the runtime firmware query from the compiled-SDK version setting. None of these fields installs firmware or certifies the runtime environment. Preserve the displayed System Software value and any supported runtime query's raw value separately. Sony's E1000 manual documents the System Information screen for the displayed version [H6].

User modules and kernel interfaces

The PSPSDK module-info definition contains a module attribute, module version, name, global-pointer value and import/export table boundaries. Its USER attribute is zero; its KERNEL attribute includes 0x1000. These are fields to inspect in the executable, not labels to infer from a source filename. The laboratory's SampleApp profile accepts an inner module attribute of exactly zero. A kernel bit is a demonstrated conflict with that profile; other unreviewed attribute combinations are not silently accepted [H9].

Thread attributes are another layer. The SDK documents a USER thread attribute and a separate VFPU attribute. They are not substitutes for module metadata, and neither removes loader restrictions. Keep the module attribute, initial-thread attributes and import-library identity as separate audit items. Copying a thread flag from a working example is not a permission model.

Do not classify a function solely from the spelling of its name. The public user headers and stubs include functions whose names begin with sceKernel. Conversely, finding a familiar name in a driver-only stub does not establish public user-mode availability. The evidence unit is the library and NID mapping, its source class and pin, and any separately obtained target-firmware evidence. Chapter 4 applies this rule to a concrete correction involving lightweight mutex and CPU interrupt names.

PRX modules and service lifecycles

PSPSDK describes PRX files as relocatable modules. Their module information and import descriptors are part of the binary interface that links an application to services. “PRX” therefore does not mean that any file can be copied next to an EBOOT and loaded like an unrestricted desktop library. The module manager documents distinct load, start, stop and unload operations, with restrictions on some loading routes [H11].

Treat each lifecycle result as a separate value. A successful load can yield a module identifier while module startup has its own result and status. A subsystem can still fail during initialization after its module is resident. Report the operation that failed instead of reducing these events to a generic “decoder unavailable.”

Optional utility modules use identifier namespaces exposed by their loaders. The older AV loader's ATRAC3PLUS identifier is 2, while the unified loader's corresponding identifier is 0x0302. Mixing the two interfaces is a caller error. The older header also records AVCODEC dependencies for ATRAC3PLUS and MPEGBASE. These declarations explain how the SDK interface is organized; they do not prove that every selected module is present and accessible on every stock firmware [H12].

A useful dependency policy

Start with the smallest known application interface and inventory the imports that actually remain in the generated PRX. Source-level intent is insufficient because runtime support and linked libraries can add dependencies. The pinned build.mak supplies default libraries and runs import fixup during the ELF build. An application with little visible code can therefore import more functions than the developer expects [S7].

For each optional feature, define what happens if it cannot initialize. A media feature should report its own failure without corrupting unrelated state. A network feature cannot be a mandatory startup dependency for a family that lacks WLAN. Add dependencies deliberately, preserve return values and test cleanup after partial initialization. Avoid a startup sequence that assumes every preceding call succeeded.

The final authority for a stock-OFW launch claim remains an observation of the identified executable in the identified device environment. The laboratory supplies strong earlier checks and a record of what remains unknown. It does not impersonate Sony's loader.

Chapter 2 — PSP Hardware Generations

Physical specifications and runtime budgets

Sony's launch material confirms different physical main-memory totals across the five families. Keep the manufacturer's units when quoting those totals. They are not measurements of SampleApp's heap, free partitions or graphics buffers.

Family	Physical main memory from Sony	Other relevant confirmed differences	Guide stock-OFW result
PSP-1000	32 MB	UMD, Duo slot, WLAN, stereo speakers, infrared listed	NOT TESTED
PSP-2000	64 MB	UMD, Duo slot, WLAN, stereo speakers, video output	NOT TESTED
PSP-3000	64 MB	UMD, Duo slot, WLAN, stereo speakers, microphone, video output	NOT TESTED
PSP Go / PSP-N1000	64 MB	16 GB internal flash, M2 slot, no UMD playback, Bluetooth, microphone	NOT TESTED
PSP Street / PSP-E1000	64 MB	UMD, Duo slot, no Wi-Fi, mono speaker, headset connector	NOT TESTED

The rows are supported by Sony's individual releases, not by assuming that later products inherit every earlier specification. PSP-1000's source is H1; PSP-2000 by H2; PSP-3000 by H3; Go by H4; and E1000 by H5.

The E1000 manual omits a main-memory number, but its launch release explicitly supplies 64 MB. This is a useful example of source discipline: an omission in one document does not justify guessing, and finding the figure in another primary source resolves the question without an unsourced inference.

RAM, embedded DRAM and VRAM are different fields

The PSP-1000 release lists 4 MB of embedded DRAM. It does not identify the whole amount as application-accessible GE VRAM. Do not copy that number into a “VRAM available to games” column. For the other families, total embedded DRAM is UNKNOWN in the cited releases.

The pinned GE header provides a runtime VRAM-size query, sceGeEdramGetSize. It documents bytes as the return unit, without promising a fixed 2 MiB value for every run. A separate setter mentions 2 MiB and 4 MiB byte values and an error for the larger selection on PSP FAT. Setter arguments are not measurements of a device's default configuration [H7].

Application free memory and largest contiguous free block also remain UNKNOWN until measured under the actual application and firmware conditions. No “24 MB usable” or “56 MB usable” number is assumed here. Physical RAM, firmware allocations, application static storage, stacks, runtime heap and service allocations are different contributors to a budget.

Installation and audio consequences

Go requires particular attention to storage. Its 16 GB flash is persistent internal storage, while M2 is a separate removable medium. It cannot play UMD content. Test internal and removable launch cases separately, and avoid assuming that one fixed volume prefix describes every device. The manufacturer's release establishes the storage types; it does not establish a universal firmware path mapping.

E1000's lack of Wi-Fi is explicit. A release that requires WLAN needs a stated family limitation or a graceful offline path. Its built-in speaker is mono, so it cannot substantiate a stereo-speaker listening test.

Headphone behavior needs its own record. For Go, Bluetooth audio or control routes need separate observations from wired output. “Audio PASS” should identify the route, channel test and duration.

PSP-3000 and Go list a microphone. Do not make the presence of that feature an unstated requirement for every family. Consumer support for a music format also does not establish that a homebrew application can directly load and use a corresponding decoder interface in its privilege context.

A model is a test condition

All cited family tables list a POWER/HOLD switch, and the SDK declares a HOLD input bit. Those facts do not verify the application's response to the switch. Physical state, reported input state, application policy and observed audio behavior belong in separate columns. A control lock can work while a resume path fails, and a stable menu says little about a long-running audio stream.

Record the complete model suffix where possible, the firmware identity, storage medium, output route and exact binary hash. Keep each unexecuted family row as NOT TESTED. A manufacturer's common display resolution does not establish identical display timing, audio behavior or firmware services across the product line.

Chapter 3 — Signing Tools Compared

“Sign” names several different transformations

Community tools use the word signing for workflows with different input formats, output formats, compression strategies and cryptographic constructions. Some wrap a plaintext PRX, some repack an existing PBP, and some create a PSN-style container around ISO content. They are not interchangeable command names.

Separate five questions when evaluating a tool: what it reads, what it writes, how it derives metadata, what its host-memory bounds actually protect, and what evidence exists for the target launch context. An upstream success claim is useful context, but the guide's compatibility matrix needs its own recorded test. The comparisons below are source observations at exact revisions; they do not claim that the alternatives were all built, benchmarked or tested on hardware.

Tool and pin	Input and output	Distinguishing behavior	Safe operational maximum established here
Legacy ebootsigner, 17d6386f034ac922f540ca78200961761b23ecae	PBP with plaintext PRX to repacked PBP	Fixed encrypted DATA.PSP template and relocation transformation	Raw PRX must be at most 4 MiB before invocation; other checks still required
SDK PrxEncrypter, e2b2b313d4175823a263c2c6cf175311e824eb77	Plain PRX to encrypted PRX	Selects fixed templates; some use gzip	UNKNOWN
sign_np, ac77d56e13b5c6c60564100699560d12ff3f425f, ELF mode	ELF/PRX to encrypted EBOOT.BIN	Derives metadata from ELF; PBP packing is separate	UNKNOWN
sign_np at the same pin, PBP mode	ISO with already signed executable to PSN-style PBP	NPUMDIMG content in DATA.PSAR	UNKNOWN
PSPPack, 934237ec4d2f0dd4f3c78071d8d2643e4603b913	PRX to minimal new PBP	Zopfli gzip and built-in PBP/SFO prefix	UNKNOWN

The selected legacy path

The legacy reference unpacks the input PBP, modifies type-7 relocation records in the extracted executable, encrypts a fixed-format body and repacks the resource sections. Its code uses KIRK1 AES-128-CBC and CMAC operations. It restores part of a template header and forges a collision for an existing data CMAC. Calling this “a new Sony signature over every application field” would misdescribe the implementation [S1].

The fixed signed DATA.PSP size is 0x553450 bytes, or 5,583,952 bytes. That source-derived format fact is not a measured SampleApp file size, because a PBP also contains its header and other sections. It is also not the permissible raw PRX input length. The legacy executable's raw ELF buffer is only 0x400000 bytes. Its complete-file read occurs before a later comparison against a larger value. The wrapper must therefore reject raw PRX input above 4 MiB before the unsafe legacy reader runs [S2].

The size check is necessary but insufficient. A small malformed ELF can contain an out-of-range section table, a relocation targeting an absent segment or an invalid module-info pointer. The laboratory validates these structures first and only admits its narrow SampleApp profile. It gives the legacy process a scratch copy, never the original source or publication destination.

The legacy templates do not create truthful payload-derived descriptions of every inner field. Keep the outer encrypted wrapper and decrypted inner module metadata separate in reports. A source-level user module needs an inner binary check; a wrapper attribute alone is not enough.

SDK PrxEncrypter

The pinned SDK implementation offers ordinary PSP and a separate pspemu-pboot family. The ordinary path chooses three fixed header pairs. Two templates enable zlib gzip; the larger one is uncompressed. Selection happens using input size before compression. These are template-selection rules, not a universal dynamically sized signer [S3].

The ordinary selector's source arithmetic produces thresholds 0x59E10, 0x68ED0 and 0x5532C0, with respective output lengths 0x59FA0, 0x69060 and 0x553450. Preserve those numbers as facts about the pinned source. Do not advertise them as certified safe input allowances: the reader is unchecked, compression and fit handling are incomplete, and five static 10 MiB buffers do not themselves validate an input.

The pspemu-pboot mode has different template and header spans. It belongs to a different investigation from the physical-PSP recipe in this guide. An option appearing in the same executable does not establish equivalent output semantics or target acceptance.

Names also need disambiguation. A project called PrxEncrypter-Mod uses a different template arrangement, 12 MiB buffers and an external 7z.exe gzip step at the examined pin. Its compression helper confuses an fread element count with a byte size. A familiar program name is not enough to identify the code whose properties were reviewed [S6].

sign_np and PSPPack

sign_np's ELF path builds PSP metadata from the executable, transforms relocations and produces an encrypted EBOOT.BIN. Its output is payload-sized rather than the legacy full template. The inspected implementation exposes several tag choices, but a flag requesting ECDSA is not proof that the selected command path generates a valid ECDSA signature. Its ELF metadata and allocation handling still require scrutiny [S4].

Its PBP path is different: the ISO must already contain a signed executable. The tool creates an NPUMDIMG block container in DATA.PSAR, with optional LZRC compression and AMCTRL-based cipher/MAC operations. DATA.PSP has metadata semantics in that format. Running this mode does not sign the executable inside the ISO automatically, and describing each ISO block as ordinary AES-CBC would oversimplify the implementation.

PSPPack builds a minimal new PBP, compresses with Zopfli and uses ELF-derived PSP metadata. This can be useful for compact homebrew, but its built-in prefix does not preserve an existing PBP's icons, sound or metadata. It is therefore not a drop-in replacement for a workflow that promises unchanged resource sections [S5].

No safe general maximum was established here for either dynamically allocated implementation. “No fixed 4 MiB array” does not mean unlimited input. Signed integer lengths, segment-count handling, allocation checks and narrowed offsets all deserve independent review.

Choosing and preserving a tool

Use the selected legacy path to reproduce this guide's exact laboratory, not because all alternatives are inferior. Choosing a different tool creates a different format contract and verification task. Preserve its repository, revision, source archive hash, build recipe and executable hash. Check the output against the relevant format rather than applying this guide's legacy-template verifier to an unrelated container.

Licenses and embedded material need component-level treatment. The pinned legacy tree has mixed notices and no single top-level license that clears the complete bundle. The SDK tool has a tool-local GPLv3 text distinct from the SDK root notice. Other dependencies retain their own notices. The companion identifies sources and provenance without distributing Sony firmware dumps or assuming that GitHub availability clears every template or constant. Consult the component records before redistribution; the observed clearance of the complete legacy bundle remains UNKNOWN.

Chapter 4 — PRX/NID Compatibility Analyzer

Inventory the binary that will be signed

The auditor begins with the plaintext PRX extracted from the validated unsigned PBP. It does not search arbitrary bytes for familiar function names. It follows the module's validated import-table boundaries and reads each descriptor using file-backed address mappings. This matters because a forged pointer or an accidental match in a string table must not turn into an apparently credible dependency report.

The narrow profile accepts the import-descriptor form produced by the laboratory: five 32-bit words, function imports and no variable imports. It bounds descriptor counts, NID arrays and function-stub spans. An unimplemented descriptor form is refused for this procedure even if another loader might accept it. Report “outside this parser profile,” rather than “invalid on every PSP.”

An import inventory should retain the library name, library version, import attributes, function count, NID value, descriptor address and source match. Names are convenient annotations; the binary's library/NID pair remains central. Two libraries can assign meaning to the same numeric identifier, and a library-name mismatch cannot be repaired by selecting a symbol with a similar spelling.

The source reference map is generated from deliberately selected public user stubs at PSPSDK revision e2b2b313d4175823a263c2c6cf175311e824eb77. It includes source paths, line numbers and fingerprints. Driver-only matches and arbitrary kernel headers are not silently admitted. Keep the map with its provenance and regenerate it from the pinned source when reviewing a release. A JSON file's claim about its own revision is not independent authentication of its contents.

Classification with an explicit evidence boundary

A matching library/NID/symbol tuple receives VERIFIED, with the explanation “SDK mapping only.” Missing mappings receive UNKNOWN. A known kernel-module attribute conflicts with the selected user profile and can be UNSUPPORTED for that stated reason. These statuses are deliberately narrower than a general homebrew compatibility database.

Property	Permissible conclusion from the inventory
Exact mapping in selected pinned public user stub	VERIFIED: SDK mapping only
No mapping in the selected source set	UNKNOWN
Inner kernel-module bit present	UNSUPPORTED by this user-module laboratory profile
Nonzero unreviewed inner attribute without that bit	UNKNOWN and refused by this profile
Export availability on the target OFW	UNKNOWN until separate evidence establishes it
Import-attribute acceptance by the target loader	UNKNOWN until separate evidence establishes it
Physical launch and application behavior	NOT TESTED until executed

A strict signing workflow may refuse UNKNOWN imports without declaring them universally unsupported. The refusal means that the publisher has not completed the required investigation. This is useful fail-closed behavior: a new dependency cannot enter the baseline release merely because a host linker supplied an import stub.

The inventory also preserves library versions and import attributes. The current mapping classification does not establish their acceptance on a particular firmware. Avoid collapsing all fields into one supported boolean. A reviewer needs to distinguish a correctly named function from a verified target export and a successfully loaded executable.

The corrected Kernel_Library example

The pinned public user stub gives the following mappings. They correct a particularly misleading class of annotation error: identifying a mutex operation as an interrupt-control operation.

Library	NID	Symbol in the pinned SDK	Status
Kernel_Library	0xBEA46419	sceKernelLockLwMutex	VERIFIED: SDK mapping only
Kernel_Library	0x15B6446B	sceKernelUnlockLwMutex	VERIFIED: SDK mapping only
Kernel_Library	0x092968F4	sceKernelCpuSuspendIntr	VERIFIED: SDK mapping only
Kernel_Library	0x5F10D406	sceKernelCpuResumeIntr	VERIFIED: SDK mapping only

The exact source is S11, Kernel_Library.S. The presence of these entries under a public user-stub source is the fact being verified. It does not establish the export or privilege behavior of every firmware, nor recommend adding interrupt-control calls to SampleApp.

A mistaken symbol annotation can send an investigation in the wrong direction. A library pulled in by threading support may legitimately add lightweight mutex imports. If those NIDs are mislabeled as CPU interrupt control, a reviewer can falsely infer an unexpected privileged dependency. Numeric values, library identity and a pinned source location prevent that error from surviving as folklore.

Investigating an unknown dependency

First, confirm that the PRX and map belong to the same report. Preserve the raw NID and exact library rather than editing the report to fit an expectation. Then inspect the link map and build options to identify which object or archive introduced the reference. Remove an unnecessary dependency at the source or link stage and rebuild; do not erase an import record directly from the binary.

If the function is required, inspect appropriate public upstream source and document the result. A match in a different library is a lead, not a substitute. A driver-only implementation can explain why the dependency deserves review without proving that a user import is allowed. When no reliable mapping exists, keep UNKNOWN and describe the missing evidence.

After changing a library choice, regenerate the full executable and inventory. Verify that the old dependency disappeared and that no new unreviewed dependency replaced it. Deleting a single line from a Makefile can change transitive imports, startup code and binary layout, so old hashes and reports cease to describe the new build.

The completed report is a dependency inventory with evidence. It is not a firmware-export dump, a behavioral emulator or a proof that arguments, concurrency and shutdown are correct. Those remaining tasks belong to the test laboratory.

Chapter 5 — Memory & Performance Engineering

Build a budget from distinct quantities

A file's length is only one part of memory planning. File-backed load bytes, zero-initialized storage, stacks, heaps, graphics resources and subsystem allocations compete for different resources. A small compressed download can expand into a much larger running application. Conversely, the legacy signer's fixed encrypted output size says little about SampleApp's runtime allocation.

The laboratory reports both file-backed LOAD bytes and static LOAD memory bytes from the executable. The difference can include storage that occupies memory without corresponding file bytes. These are binary-analysis figures, not observed peak memory use. The parser's 32 MiB static-LOAD ceiling is an explicit policy for admitting this small example; it does not mean that every admitted image leaves enough memory for its heap and stacks.

A BUDGET IS MORE THAN A FILE SIZE

Static LOAD
153,988 B measured

Heap request
2,048 KiB linked

Stack request
32 KiB linked

Add worker stacks, runtime/service allocations and a safety reserve.
Graphics capacity is separate: query and budget actual buffers.
No physical peak-use or available-RAM claim is made.

Plan a budget with named categories and a reserve. Keep assumptions visible until measurements replace them.

Budget item	Evidence to collect	Common mistake
Static executable storage	LOAD file and memory sizes, link map	Treating file size as total resident demand
Runtime heap	Requested size, allocation failures, observed use	Assuming all installed RAM belongs to malloc
Thread stacks	Number, configured sizes, worst-case call behavior	Large temporary arrays on a small stack
Graphics resources	Queried GE capacity, buffer sizes, allocation layout	Confusing total embedded DRAM with accessible VRAM
Services and media buffers	Initialization changes, stream buffers, cleanup	Forgetting allocations outside the application's own allocator
Safety reserve	Defined workload and remaining capacity	Budgeting exactly to an unmeasured limit

The appropriate reserve is an application decision supported by testing, not a universal number supplied by this handbook. Keep performance and memory peaks NOT MEASURED where measurements have not been obtained.

Heap, stack and allocation lifetime

Choose allocation lifetime deliberately. Long-lived objects should have clear owners and a cleanup path. Temporary work buffers should be bounded by validated input dimensions. A media file or image header must not be allowed to request arbitrary memory merely because the file itself is small.

Check every allocation that can fail. Propagate the failure to a stage-specific error, release resources already acquired and leave the program in a defined state. A null pointer dereference after an unchecked allocation often looks like a mysterious firmware hang to an end user. The actual fault can be ordinary application error handling.

Avoid repeatedly allocating large buffers in a rendering or streaming loop. Reuse a bounded buffer when the workload permits, and record its capacity separately from the current data length. If a contiguous allocation fails while total free memory appears sufficient, inspect the largest free block and fragmentation. The SDK provides distinct total-free and largest-free-block queries [H8].

Stacks deserve explicit attention because their failures may corrupt unrelated state before becoming visible. Account for nested calls, library routines and callback paths. Do not move a large heap buffer onto the stack merely to avoid checking an allocation. Keep callback work short enough that its memory and synchronization requirements can be understood.

Threads, CPU and VFPU

The pinned thread header declares a VFPU-access attribute separately from USER mode [S12]. Use the documented attribute when code in a thread requires that interface, and review the actual worker thread rather than assuming the main thread's settings apply everywhere. The presence of a flag is not a benchmark, and enabling it does not automatically make scalar code faster.

Treat CPU-frequency changes as a separate experiment with explicit API results and device conditions. Sony's hardware tables advertise a CPU frequency range, while the SDK power header documents a clock-setting interface and parameter relationships [S13]. Those facts do not show the rate used by a specific run or that a request was accepted. Log requested and observed conditions where supported; do not publish invented speedups.

Measure a workload that represents the real application. A one-frame display test cannot establish sustained decoder throughput. A timing result without the build hash, workload, emulator or hardware identity, and relevant settings is difficult to reproduce. Keep warm-up, duration and repeated-run variability visible when reporting performance.

Concurrency failures also appear as memory or firmware problems. Keep ownership of shared buffers clear, use synchronization with known mappings and avoid waiting while holding a lock that another required thread needs. Distinguish a blocked worker, a callback that is never serviced and a failed allocation. They require different evidence and different fixes.

Graphics and stream-buffer arithmetic

Calculate capacities before allocating. Width, height, stride, bytes per pixel and buffer count are separate factors. A conceptual double buffer with stride 512, height 272 and four bytes per pixel needs 1,114,112 bytes before depth, textures or other storage. This is arithmetic illustrating a budget, not a measured allocation or a claim about a mandatory PSP framebuffer format.

Use checked arithmetic for dimensions and counts. Verify that multiplication and alignment cannot wrap the chosen integer type. Keep the original logical dimensions beside the padded stride so that a later copy does not use the wrong length. Apply the same discipline to audio frames, compressed packets and ring-buffer indices.

A program that runs once under an emulator can still fail after repeated resource creation or a suspend/resume cycle. Include repeated open/close, error cleanup and long-duration scenarios in the eventual hardware plan. Until then, retain NOT TESTED rather than projecting a startup observation onto the whole application lifetime.

Chapter 6 — EBOOT Forensics and Safe Signing

Read the PBP as a bounded container

The PBP used by this laboratory begins with a 40-byte header: magic, version and eight section offsets. The offsets identify section starts; the next offset, or the end of the file for the final section, determines a length. Equal adjacent offsets represent an empty section. The pinned PSPDEV packer shows the expected section order and the construction of those offsets [S9].

Index	Section	Role in this workflow
0	PARAM.SFO	Typed metadata and declared application properties
1	ICON0.PNG	Optional icon resource
2	ICON1.PMF	Optional animated resource
3	PIC0.PNG	Optional picture resource
4	PIC1.PNG	Optional picture resource
5	SND0.AT3	Optional menu sound resource
6	DATA.PSP	Plaintext PRX before signing; legacy encrypted payload afterward
7	DATA.PSAR	Additional content; interpretation depends on the format

The narrow parser requires the expected magic and version, a first section beginning immediately after the header, ordered offsets and every section within the file. Its 64 MiB whole-container ceiling is a laboratory resource policy, not a Sony file-format maximum. A header-only check does not validate PARAM.SFO or DATA.PSP.

Do not assign one meaning to DATA.PSAR across every PBP family. It can be empty in a small homebrew package, while sign_np's PSN-style path uses it for NPUMDIMG. Similarly, DATA.PSP semantics depend on the format. This guide's plaintext-PRX and fixed-template expectations apply to its selected legacy path.

PARAM.SFO deserves its own parser

SFO is not a string search. Its header locates an entry table, key table and value storage. Each entry identifies a key, type, current length, allocated maximum and relative value offset. Parsing must validate those relationships before interpreting a value. Finding “MEMSIZE” somewhere in a byte stream does not establish which entry owns it or what its type is.

The laboratory checks entry count, table order, alignment, bounded key terminators, duplicate keys, value lengths, value-storage overlap and supported data types. Integer entries have an exact width; strings need a valid terminator and encoding. A malformed entry must fail before its bytes become a configuration value.

The SampleApp contract then checks these typed values:

Key	Required value	Why it is checked
TITLE	SampleApp	Connects the package metadata to the example
CATEGORY	MG	Fixes the chosen application profile
BOOTABLE	Integer 1	Prevents accidental metadata drift

Key	Required value	Why it is checked
MEMSIZE	Integer 0	Makes the laboratory's memory-request policy explicit
PSP_SYSTEM_VER	String 6.60	Fixes the declared target metadata without claiming tested compatibility

MEMSIZE zero is a deliberate baseline policy. The pinned build.mak has memory-expansion logic with CFW-specific comments and can add MEMSIZE automatically. Set the example's intended build option explicitly, then inspect the generated SFO. Do not assume a Makefile variable achieved its purpose. PSP_SYSTEM_VER is likewise a typed metadata declaration; setting it to 6.60 does not prove launch on 6.60 or 6.61 [S7, S8].

The independent crypto verifier does not make every shared parser independently implemented. Its cryptography uses a different implementation from the signer, while structural and semantic checks still have their own code paths and limits. Describe that division accurately. Rechecking SFO fields and preserving unchanged resource bytes closes specific gaps; it does not turn a shared parsing function into an independent full loader.

Validate ELF and PRX before legacy code reads them

The admitted executable is ELF32, little endian, MIPS and PSP PRX type 0xFFA0. Ordinary ELF executables and unrelated ELF files are outside this profile. A renamed extension does not change those fields.

Program and section header tables must fit in the file with the expected entry sizes and bounded counts. Every file span must be checked using arithmetic that cannot wrap. LOAD segments need coherent file and memory sizes, valid address ranges, acceptable alignment and nonoverlapping admitted memory intervals. The entry point must lie in an executable file-backed LOAD range. These conditions protect the parser and legacy workflow; they are not a complete emulation of Sony's loader.

Virtual addresses need translation through validated file-backed LOAD segments. They are not file offsets. A pointer into zero-filled memory does not identify bytes that can be read from the input file. Reject unmapped or ambiguous address translations. Apply these checks to module information, strings, import descriptors, NID arrays, function stubs and export ranges.

The module-info section must be unique in the selected profile, large enough, file-backed and consistently mapped. Check its bounded name, exact SampleApp identity and attribute. Preserve the import/export boundaries rather than allowing a later function to reinterpret unchecked values.

Relocation sections are also executable input to the legacy transformation. Validate record alignment, type encoding, referenced LOAD indices and target offsets. Refuse relocation storage overlapping ELF headers or tables, mapped LOAD bytes, another file-backed section or another relocation span; the legacy loop must not mutate metadata that it rereads. The narrow profile refuses encodings it does not implement. Its deliberate restriction is preferable to treating arbitrary relocation bytes as harmless data.

The allowed relocation change

The pinned legacy relocation pass changes records whose low relocation-type byte is 7 by clearing that byte. It does not justify arbitrary modifications elsewhere in the PRX [S10]. The verifier independently computes the expected transformation from the validated original, then compares the complete corresponding plaintext PRX prefix after decryption.

That full comparison matters. Seeing an ELF magic value after decryption only proves that a few bytes resemble an executable header. Checking one sample instruction leaves most of the program unchecked. The expected transformed bytes, module metadata and imports must remain connected to the original build identity.

The remainder of the fixed legacy body is format-specific padding and transformation space. Verification should follow the exact pinned layout, lengths and authenticated regions rather than guessing that all decrypted bytes are part of the original file. A reported relocation count is a measured result only when generated from the actual artifact.

Isolate the signer and protect the original

The safe procedure is designed for trusted, audited SampleApp build inputs. It reduces accidental overwrite and malformed-input exposure. It is not a hostile-code virtual machine or a promise of complete operating-system sandboxing.

1. Snapshot a regular input file within the container-size policy and reject an obvious symlink input. Check that its size and relevant file metadata remain stable while reading.
2. Validate PBP, typed SFO, PRX, module policy and dependency inventory before starting legacy code.
3. Require a distinct, previously nonexistent output path in a trusted existing parent directory. Reserve a requested report exclusively before any signing or packaging operation; report/output aliases and unavailable report destinations must prevent that operation.
4. Fingerprint the exact signer executable and compare it with the expected hash.
5. Create a unique temporary job directory and give the signer only its scratch input and scratch output names.
6. Apply process time, memory, file-size and descriptor limits in the POSIX laboratory environment.
7. Require a successful exit and a regular candidate output, then run the independent checks.
8. Recheck the scratch input and original input identity before publication.
9. Publish using exclusive creation, so an existing destination is never opened for replacement.

The job directory avoids collisions between concurrent or repeated legacy runs that use fixed internal filenames. The original input never becomes the signer's writable working file. A refusal leaves the publisher to inspect evidence rather than deciding silently which older file to overwrite.

Exclusive creation protects existing destinations, but a system crash can still leave an incomplete newly created file. Verify the published hash before distributing it. The procedure also assumes trusted parent directories and source tooling; do not describe it as protection against every filesystem race or a malicious signer replacing itself during execution. Stronger host isolation belongs to the documented container setup.

Independent cryptographic cross-validation

For this pinned legacy format, the verifier checks that non-DATA.PSP sections are unchanged, the wrapper matches its exact template, KIRK mode and layout fields are expected, and all relevant spans fit the fixed body. It retrieves template material from exact upstream source blobs rather than accepting an arbitrary similarly named file.

It uses PyCA AES-CBC and CMAC operations instead of calling the signer's C crypto. It checks both the header CMAC and data CMAC, decrypts the payload and compares the complete expected transformed PRX prefix. It reparses the transformed executable and confirms that the transformation did not change module or import metadata.

These checks provide independent cryptographic implementation and byte-level continuity between the unsigned and signed artifacts. They do not certify the provenance of every historical template, establish a fresh Sony-issued signature, or execute Sony's firmware checks. The report must retain “offline crypto and structural SampleApp profile” as its scope and physical OFW as NOT TESTED.

Extract components without losing provenance

Inspection should create new files in a dedicated output directory using fixed section names. Preserve the source PBP hash and each extracted section's offset, size and hash. Never use an unchecked embedded string as a filesystem destination, and do not let an extraction overwrite an existing original.

Keep unsigned and signed extractions separate. A DATA.PSP named in one directory is ambiguous without the parent PBP identity and transformation stage. If a forensic experiment mutates a section, create a separate copy and retain the mutation description. This makes a negative control reproducible and prevents a deliberately damaged test file from being mistaken for a release candidate.

Chapter 7 — Reproducible Build Laboratory

Preserve the environment as an input

A reproducible build is a claim about identified inputs producing identified outputs. “Built with PSPDEV” is too broad: a moving image tag, a changed SDK, a different compiler or a previously generated PARAM.SFO can change the result. Record the source commit and dirty-tree state, tool executable versions and hashes, dependency locks, image identity, commands and output hashes.

The laboratory uses the PSPDEV project image pinned by digest, with separately locked host packages and Python dependencies. Its exact base identity is pspdev/pspdev@sha256:54895e6f5afb71b8f4f6915ee6d5023e1e087ca7afdf2dcb1d7a694a5233e45f. A digest identifies the selected base bytes; additional package installation and the guide image build still need their own records. Keep the final image identifier and lock files with the evidence.

A reference source checkout and the SDK installed in a development image are not automatically the same revision. This guide records the e2b2b313d4175823a263c2c6cf175311e824eb77 SDK source used for the NID and build-rule review. The hardware API research also cites a separately pinned 6ef2b21dc440ef8d4125f000301faaa2a470975e header snapshot. Preserve those distinctions. Do not replace all pins with one convenient date or assume that source used for annotation was used to build every binary.

Windows 11, WSL2 and Linux

On Windows 11, the documented laboratory commands run in the POSIX environment provided by the chosen WSL2 or Docker setup. Keep the shell and path convention explicit. A Windows path, a WSL mount path and a container path can refer to related files without being interchangeable command arguments.

Use a dedicated task directory and an explicit bind mount. Record where source is read, where outputs are created and which directories permit execution. Avoid installing build dependencies into the user's unrelated projects. A working directory containing stale outputs makes the later comparison less meaningful even if the compiler itself is pinned.

On Linux, use the same container recipe and locks when reproducing the published run. A native build can be an additional experiment, but record it as another environment rather than silently substituting it for the pinned recipe. Filesystem case behavior, executable permissions, paths and line endings are common sources of accidental differences.

Container isolation and reproducibility are related but separate. Offline execution, a read-only root, dropped capabilities and a task-specific writable mount restrict the job environment. They do not prove that a file is a valid EBOOT. Conversely, deterministic output bytes do not prove that the host setup provides adequate isolation. Keep both contracts visible.

Audit main.c and the Makefile together

The example's source declares its module identity and privilege profile, initializes its visible behavior and provides the exit path. Its Makefile chooses how that source becomes a PRX and how metadata becomes a PBP. Reviewing only one file leaves half the contract unchecked.

The complete current SampleApp source follows. It is the same example supplied in the companion; its results belong to the actual artifacts identified in the measured report.

```
/* SPDX-License-Identifier: MIT */
#include <pspkernel.h>
#include <pspdebug.h>
#include <pspctrl.h>
#include <pspge.h>
```

```
PSP_MODULE_INFO("SampleApp", 0, 1, 0);
PSP_MAIN_THREAD_ATTR(PSP_THREAD_ATTR_USER);
PSP_MAIN_THREAD_STACK_SIZE_KB(32);
PSP_HEAP_SIZE_KB(2048);

volatile unsigned sample_ready, sample_heartbeat, sample_exit_reason;
volatile unsigned sample_free_total, sample_free_max, sample_ge_bytes;
static int quit;

static int exit_callback(int arg1, int arg2, void *common)
{
    (void)arg1; (void)arg2; (void)common;
    sample_exit_reason = 1;
    __atomic_store_n(&quit, 1, __ATOMIC_RELAXED);
    return 0;
}

int main(void)
{
    SceUID cb = sceKernelCreateCallback("SampleExit", exit_callback, NULL);
    if (cb < 0 || sceKernelRegisterExitCallback(cb) < 0) {
        sceKernelExitGame();
        return 1;
    }
    sceCtrlSetSamplingCycle(0);
    sceCtrlSetSamplingMode(PSP_CTRL_MODE_DIGITAL);
    pspDebugScreenInit();
    pspDebugScreenPrintf("SampleApp - User Mode\n");
    pspDebugScreenPrintf("Physical stock OFW: NOT TESTED\n");
    pspDebugScreenPrintf("CROSS: application exit\nHOME: exit callback\n");
    sample_free_total = sceKernelTotalFreeMemSize();
    sample_free_max = sceKernelMaxFreeMemSize();
    sample_ge_bytes = sceGeEdramGetSize();
    sample_ready = 1;
    while (!__atomic_load_n(&quit, __ATOMIC_RELAXED)) {
        SceCtrlData pad = {0};
        if (sceCtrlPeekBufferPositive(&pad, 1) > 0 && (pad.Buttons & PSP_CTRL_CROSS)) {
            sample_exit_reason = 2;
            __atomic_store_n(&quit, 1, __ATOMIC_RELAXED);
        }
        ++sample_heartbeat;
        sceKernelDelayThreadCB(20000);
    }
    sceKernelExitGame();
    return 0;
}
```

Check callback setup, every important return value and the relationship between the main loop and exit handling. A callback identifier must be created successfully before registration is assumed. A worker thread that should service callbacks needs a callback-aware waiting strategy. The SDK's exit and thread documentation gives the relevant interface; physical HOME behavior still needs a device test [S12].

The matching Makefile follows. Its exact flags are part of the build identity.

```
TARGET = SampleApp
OBJS = main.o
CFLAGS = -O2 -G0 -Wall -Wextra -Werror -ffunction-sections -fdata-sections
LDFLAGS = -Wl,--gc-sections,-Map,build/SampleApp.map
LIBS =
BUILD_PRX = 1
PSP_FW_VERSION = 660
PSP_LARGE_MEMORY = 0
PSP_EBOOT_TITLE = SampleApp
SFOFLAGS += -s PSP_SYSTEM_VER=6.60
EXTRA_TARGETS = EBOOT.PBP
PSPSDK := $(shell psp-config --pspsdk-path)
include $(PSPSDK)/lib/build.mak
```

The pinned build.mak selects PRX specs when BUILD_PRX is enabled, runs psp-fixup-imports unless explicitly disabled, invokes psp-prxgen and packages the resulting PRX with SFO and resources. It also

supplies default libraries, compiler firmware macros and metadata behavior. Inspect the expanded build output rather than assuming that a short Makefile is the whole recipe [S7].

Set the SampleApp memory and system-version policies explicitly, then inspect the generated SFO. Do not rely on a stale PARAM.SFO target that make considers up to date after flags change. A clean isolated build directory is simpler to reason about than carefully guessing which generated file needs deletion.

The optional ENCRYPT rule in the examined build.mak can invoke PrxEncrypter with the same PRX path as input and output, and the recipe has its own error-handling behavior. This guide's safe legacy path does not use that in-place route. Preserve the unsigned PRX and PBP, then sign a scratch snapshot into a distinct new output. This is a workflow choice made to keep original evidence available.

Build twice without sharing outputs

Create two empty build directories from the same identified source snapshot. Give each the same pinned environment and commands, but do not copy object files, generated SFOs or PRXs from the first directory into the second. Retain logs separately. The second run should be able to expose an omitted dependency or a hidden stale artifact.

Compare the stages that matter: ELF, plaintext PRX, generated SFO, unsigned PBP and any deterministic signed result. Retain individual hashes rather than reporting only that two directories look similar. When an output differs, preserve both and locate the first differing stage. A debug-path change has a different implication from a changed executable section, but neither should be erased before investigation.

Two identical builds support repeatability in the documented environment. They do not automatically establish reproducibility across every host, compiler version or filesystem. State the tested comparison precisely. If the builds differ, the release can still report useful evidence while keeping reproducibility FAIL or UNKNOWN until the difference is explained.

Executed commands and measured artifacts

The following block is generated from the actual laboratory workflow. Its paths, tool identities and command results take precedence over illustrative terminology elsewhere in the prose. Run it in the stated directory and environment; do not substitute an unrelated signer with the same filename.

These replay commands use the recorded WSL2 shell and Docker Desktop. Extract the companion into a new `source` directory inside an otherwise empty laboratory directory. On Linux, use `docker` in place of `docker.exe`. Paths below are container paths except in the first setup block. Choose fresh build/report/archive destinations; do not run these commands over the supplied evidence.

Fetch reference sources online before starting the offline build job. The fetcher checks archive and individual-file hashes; it never replaces an existing source directory. Its fresh-download path and cached verification were both executed successfully. An attempted download inside the offline container failed as expected; that environment log is preserved, not called a successful fetch.

```
# Host shell, in the newly extracted source directory.
python3 -B tools/fetch_sources.py ../upstream
python3 -B tools/fetch_sources.py ../upstream --check-existing
docker.exe build -t psp-guide-lab:locked .
```

Mount the dedicated laboratory directory as `/lab`, with this source at `/lab/source`. The measured job used a network-disabled container, read-only root, all capabilities dropped, no-new-privileges, a 768 MiB memory limit, PID limit 64, and a 256 MiB noexec/nosuid `/tmp` tmpfs. The complete host setup is in REPRODUCE.md; the commands below run inside that container in `/lab/source` as its unprivileged UID 1000. The exact resulting image ID is recorded in the build manifest. Recreating package repositories later may fail if a locked version is no longer retained; do not silently replace a pin.

```
# Inside the locked container; fresh paths only.
mkdir /lab/test-tmp
TMPDIR=/lab/test-tmp python -B -m unittest discover -v
make -C /lab/upstream/ebootsigner \
  CFLAGS='-O2 -std=gnu99 -Wall'
sha256sum /lab/upstream/ebootsigner/ebootsign
mkdir -p /lab/build-a/build /lab/build-b/build
```

```
cp sample/main.c sample/Makefile /lab/build-a/
cp sample/main.c sample/Makefile /lab/build-b/
make -C /lab/build-a -j2
make -C /lab/build-b -j2
```

The measured legacy signer hash is below. Require it before replaying the pinned signing path. A different host binary hash requires an explained new environment record; it is not permission to bypass the check. The original build completed with two retained legacy C array-parameter warnings. SampleApp itself compiled with -Werror.

```
SIGNER_SHA=0474c79ef666947bf16f2695680a380021ccb4be8c78bfca97bb7f871cd2077c
python -B tools/preflight_pbp.py /lab/build-a/EBOOT.PBP \
  --nid-map references/user-nids.json \
  --report /lab/preflight-a.json
python -B tools/nid_audit.py /lab/build-a/SampleApp.prx \
  --nid-map references/user-nids.json \
  --report /lab/imports-a.json
python -B tools/safe_sign.py \
  /lab/build-a/EBOOT.PBP /lab/build-a/EBOOT-signed.PBP \
  --signer /lab/upstream/ebootsigner/ebootsign \
  --signer-sha256 "$SIGNER_SHA" \
  --sources /lab/upstream/ebootsigner \
  --nid-map references/user-nids.json \
  --report /lab/signed-a.json
python -B tools/safe_sign.py \
  /lab/build-b/EBOOT.PBP /lab/build-b/EBOOT-signed.PBP \
  --signer /lab/upstream/ebootsigner/ebootsign \
  --signer-sha256 "$SIGNER_SHA" \
  --sources /lab/upstream/ebootsigner \
  --nid-map references/user-nids.json \
  --report /lab/signed-b.json
```

Run the separate verifier, rather than relying only on the wrapper's report. Its source directory is the fetched signer reference; its cryptography is PyCA, not the C signer's implementation. The NID map describes public SDK mappings, not a stock-firmware export census.

```
python -B tools/cross_validate.py \
  /lab/build-a/EBOOT.PBP /lab/build-a/EBOOT-signed.PBP \
  --sources /lab/upstream/ebootsigner \
  --nid-map references/user-nids.json \
  --report /lab/independent-a.json
python -B tools/inspect_sample_elf.py /lab/build-a/SampleApp.elf \
  --report /lab/linked-elf.json
cmp /lab/build-a/SampleApp.elf /lab/build-b/SampleApp.elf
cmp /lab/build-a/SampleApp.prx /lab/build-b/SampleApp.prx
cmp /lab/build-a/PARAM.SFO /lab/build-b/PARAM.SFO
cmp /lab/build-a/EBOOT.PBP /lab/build-b/EBOOT.PBP
cmp /lab/build-a/EBOOT-signed.PBP /lab/build-b/EBOOT-signed.PBP
python -B tools/negative_controls.py \
  --unsigned /lab/build-a/EBOOT.PBP \
  --signed /lab/build-a/EBOOT-signed.PBP \
  --sources /lab/upstream/ebootsigner \
  --nid-map references/user-nids.json \
  --output /lab/negative-controls
```

The supplied artifacts and reports document these operations on the actual example. Raw sizes 4 MiB minus one, exactly 4 MiB, and 4 MiB plus one are preflight padding controls only. Oversized or malformed controls never reach the legacy signer or a device.

Finally, create a new staging tree with README.md, LICENSE.txt, VALIDATION.md, applicable notices, and PSP/GAME/SampleApp/EBOOT.PBP copied from the checked signed output. The supplied installation example is packages/SampleApp-Install.zip. For a fresh replay staging directory:

```
python -B tools/package_release.py \
  /lab/stage-install /lab/SampleApp-Install.zip \
  --report /lab/install-package.json
```

This is a new-archive operation. It includes root documentation and every staged payload, then checks CRC, exact members and all payload hashes. Its internal SHA256SUMS excludes itself; the external

package inventory hashes that manifest explicitly. The existing output and report paths are refused before packaging can run. Disk failure or a crash is still a separate condition: verify the final artifact hash before installation or distribution.

The report below supplies newly obtained sizes, hashes and comparison results. Source-derived constants elsewhere in the handbook are labeled as such. Runtime performance, physical free memory and device compatibility remain unmeasured or untested unless their own rows contain observations.

Artifact	Bytes	A/B identity
EBOOT.PBP	159,414	PASS
SampleApp.prx	159,058	PASS
SampleApp.elf	1,046,908	PASS
EBOOT-signed.PBP	5,584,308	PASS

Full SHA-256 identities (both builds):

EBOOT.PBP

```
dd1bc484679a96cac8aed994813242099a110b95d6c8bf9efcb37e6607c77c88
```

SampleApp.prx

```
de998cea467ae8671036edeebc0fa5b86069b16252a22e654f0536f52dd67b29
```

SampleApp.elf

```
b27a9bd0c63201802b8e26e153e7990e1fdd1dd9eb7006da05e32afe2a77b617
```

EBOOT-signed.PBP

```
d82c452f1b8200fab75d3bd54878da5dea7b364fc8119e80cb868bee5fcf6f47
```

Measured field	Value	Scope
Plaintext PRX	159,058 bytes	Build A and B
Signed DATA.PSP	5,583,952 bytes	Fixed-format body in actual output
File-backed LOAD storage	128,224 bytes	Static binary analysis
Static LOAD memory	153,988 bytes	Static binary analysis; excludes heap/stacks
Type-7 relocation changes	0	Actual input
Imports	12 libraries / 54 functions	All mapped to selected SDK user stubs
Inner module / main thread	0 / 0x80000000	Module profile / linked USER thread attribute
Linked stack / heap request	32 KiB / 2,048 KiB	pyelftools parameter audit
SFO memory / version	MEMSIZE=0 / PSP_SYSTEM_VER=6.60	Typed metadata, not compatibility
Physical peak RAM / CPU / FPS	NOT MEASURED	No physical run

The generated SFO also contains DISC_ID=UCJS10041, DISC_VERSION=1.00, PARENTAL_LEVEL=1 and REGION=32768. These are SDK generator defaults, not an assigned Sony publishing identity for this example. All generated SFO bytes are retained.

The final PSPDEV-derived image identity, exact tool hashes, linked parameters and source hashes are in environment/build-manifest.json. The source revision of the SDK installed inside the base image is UNKNOWN; the reference checkout pins describe source review, not a claim about that installed SDK revision.

A measurement without its artifact is difficult to audit. Keep the unsigned and signed files, link map, compiler output and machine-readable reports together in the companion when their distribution is permitted. Never publish an old SampleApp checksum beside a newly rebuilt example. A later source edit invalidates the association even if the application still prints the same text.

Chapter 8 — Automated Test Laboratory

Tests should challenge the claimed property

Host tests are especially useful for the laboratory's parsers, policy gates, overwrite protection and cryptographic verification. They run earlier and can exercise malformed inputs that should never reach a physical device. A meaningful test asserts the expected rejection or verified property, not merely that a script exits without a Python traceback.

Keep fixtures small and describe how they were created. A deliberately malformed relocation table is easier to understand when the exact mutated field and expected failure stage are recorded. Negative controls belong in temporary copies, separate from release staging. Never corrupt the only preserved input to demonstrate that corruption is detected.

A useful test family includes both a valid baseline and one changed condition. If a parser rejects every file, its negative tests can all pass while the tool is useless. If a verifier only accepts an output produced by its own unchecked writer, its agreement may be circular. Include independently constructed malformed cases and compare expectations with the actual source contract.

Structural and metadata controls

For PBP handling, exercise truncated headers, wrong magic or version, offsets before the allowed start, descending offsets, values past end of file and boundary-sized empty sections. Check both the reported failure and the absence of unsafe extraction or signer invocation.

For SFO, exercise missing required keys, duplicate keys, wrong types, invalid lengths, unterminated strings, invalid offsets, overlapping values and changed MEMSIZE or PSP_SYSTEM_VER. A field that is present with the wrong type must not satisfy a semantic profile. These controls specifically challenge the gap left by checking metadata only through a packer's successful exit.

For ELF/PRX, exercise truncated program or section tables, unexpected entry sizes, address overflow, out-of-file spans, invalid section names, overlapping LOAD ranges and an entry point outside executable file-backed storage. Challenge module-info pointers and import arrays independently. A structure can be internally plausible at one level while pointing outside the next.

For relocations, include invalid record alignment, unsupported encodings, absent segment indices and targets beyond file-backed bytes. Verify that rejection occurs before the legacy tool runs. Preserve type-7 transformation controls separately from general ELF-bounds controls so that a correct mutation cannot hide a permissive parser.

Cryptographic and continuity controls

Begin with a signed artifact whose exact input and tool identity are known. Corrupt a byte in an authenticated region and require the appropriate CMAC or structural check to fail. Corrupt the wrapper or truncate the body and require layout rejection. Change a non-DATA.PSP resource and require continuity checking to reject the pair.

Challenge the plaintext comparison as well. A decrypted ELF header alone is insufficient: the verifier must reject a mismatch deeper in the transformed executable. Preserve the expected relocation adjustment and confirm module/import metadata continuity. A source blob with a different template must fail the pin check before it is used.

The verifier's independent cryptographic implementation reduces the chance that the signer's C crypto and checker share the same implementation bug. It does not remove every shared assumption about layout or template material. Keep code review, exact pins, full byte comparison and negative controls as complementary evidence.

Process safety and archive controls

Use controlled executable fixtures to test process failure, timeout and missing output without asking the legacy signer to process dangerous files. Verify that a nonzero child exit leaves the original unchanged and publishes no candidate. Exercise existing destinations, aliases, symlinks where applicable, signer hash mismatch and input changes detected by the snapshot procedure.

Host environment failures need honest classification. A fixture located on a noexec filesystem can fail before its intended behavior runs. That is evidence about the test setup, not a successful exercise of the fixture's target condition. Preserve the failed log, correct the setup and record the rerun separately.

For archives, test fresh creation, refusal of an existing destination, required root documentation, case collisions, unsafe member names, symlinks and manifest disagreement. Verify both member content hashes and ZIP integrity. A CRC catches some accidental damage; it does not replace SHA-256 identity or an explicit expected member set.

PPSSPP integration

An emulator is a valuable integration environment for seeing whether a build starts, displays its expected screen, handles modeled input and reports obvious runtime errors. Record the PPSSPP executable identity, command, relevant configuration, log, artifact hash and observation method. A screenshot becomes more useful when it is connected to those details.

Keep execution distinct from mere file recognition. An emulator process can return a host exit code without demonstrating every application assertion. Define what counts as reaching the intended screen or completing the scenario, and preserve evidence for that criterion. Automated capture and logs should describe their own limits.

PPSSPP does not become a Sony OFW loader because it can open a signed PBP. It may model behavior differently or omit enforcement found in a physical environment. Report emulator results in their own rows. Use them to find defects and guide device tests, not to fill an untested stock-OFW matrix with PASS.

Directory-name experiments in PPSSPP are likewise emulator observations for exact names, encoding, paths, configuration and build. A successful long name does not establish a universal physical PSP limit. A failed name can reflect the host filesystem or installation layout. Preserve those conditions and avoid converting one observation into a platform rule.

Evidence ready for review

The matrix below is generated from the actual laboratory records. It keeps executed host checks, emulator observations, unmeasured performance and untested hardware separate.

Executed check or open case	Result	Evidence / boundary
Final native host parser, crypto, NID and safety suite	PASS: 38 / 38	host-final.log; original32-test run and noexec error retained
Real-artifact negative CLI controls	PASS: 13 / 13	negative/summary.json; expected rejections retain FAIL reports
Two independent build directories	PASS: 4 / 4 identical artifacts	reproducibility.json; also SFO comparison in build manifest
Isolated signer A and B	PASS	Exact input/tool hashes; original snapshots unchanged
Separate crypto/profile CLI	PASS	sample-independent-cli.json
Linked ELF parameter audit	PASS	linked-elf.json; parser independent of guide PRX parser
Cold PPSSPP direct launches	PASS: 6 / 6	attempt-005/report.json; same signed hash

Executed check or open case	Result	Evidence / boundary
Live input and exit in PPSSPP	PASS: 6 / 6	CROSS -> reason 2 -> exit import -> game.quit
Physical PSP stock OFW 6.60 / 6.61	NOT TESTED	All five physical families
Physical XMB / HOME / suspend-resume	NOT TESTED	No physical observation
Physical peak RAM / sustained performance	NOT MEASURED	No physical profiling
Target firmware exports / attribute acceptance	UNKNOWN	SDK mapping is insufficient

PPSSPP 1.20.4 ran in interpreter mode with software Mesa and emulated PSPModel=0. Each cold run observed SampleApp ready=1, an advancing heartbeat, and the same EBOOT hash. The debugger sent CROSS, observed exit_reason=2 at the real exit import (0x08820B28), then observed game.quit and a stopped CPU. This tests the application exit path; the physical HOME callback remains NOT TESTED.

ASCII folder bytes	Direct-launch result	Publisher rule
1	PASS in PPSSPP only	Admit
9	PASS in PPSSPP only	Admit
31	PASS in PPSSPP only	Admit
32	PASS in PPSSPP only	Admit
33	PASS in PPSSPP only	Refuse
64	PASS in PPSSPP only	Refuse

Names were A, SampleApp, then repeated A for the other lengths. The 33- and 64-byte diagnostic paths deliberately bypass the publisher rule. None of these direct launches tests Sony XMB directory discovery or establishes a Sony name-length limit.

Emulator-only ready snapshot	Bytes
Total free memory	22,865,408
Largest free block	22,603,264
GE-reported memory	2,097,152

These are observed emulator values for this build and configuration, not physical PSP specifications, application peak use, leak tests or performance results.

Earlier runtime attempts 001-004 remain FAIL in the evidence. They exposed debugger discovery/configuration, startup timing and an incorrect expectation of a ticketed cpu.resume response. Attempt 004 already reached the live application exit path; the harness timed out afterward. The corrected harness in attempt 005 awaits the documented asynchronous event. SampleApp was unchanged. The host suite also retains its initial noexec-fixture ERROR and the corrected 32-test run. See evidence/runtime/README.md for the chronology.

After the saved runs, the CLI report-path review added focused regression tests. All six focused tests and the complete 38-test suite passed; see evidence/cli-regression-final.log, evidence/host-final.log and TECHNICAL_AUDIT.md. The findings and their regression failures are preserved. They do not replace or erase the original 32-test evidence.

A useful evidence record includes the check name, status, artifact hashes, tool/environment identity, command or procedure, relevant output and interpretation. Preserve real FAIL rows and the reason for a later rerun. A corrected experiment can add a PASS without rewriting the earlier history into an uninterrupted success.

Physical tests should eventually add model, firmware state, launch medium, screen-entry behavior, HOME exit, HOLD transitions, audio route and suspend/resume observations. These rows are NOT TESTED in the absence of such execution. Automated host tests can make the release much more inspectable while leaving that boundary honestly open.

Chapter 9 — Common PSP Development Mistakes

Diagnose the stage, not the folklore

“Corrupted Data” is an observation to record, not a unique diagnosis. It can motivate checks of installation, PBP structure, metadata, executable content and launch context. Do not choose one explanation solely because it fixed somebody else's unrelated package.

Begin with the release hash and installation tree. Confirm that the file on storage matches the checked release artifact, that EBOOT.PBP is at the intended level and that an extra extracted directory has not shifted the path. Then inspect the package using the host tools. Only after these checks should a new signing or dependency hypothesis replace the existing evidence.

Preserve the exact error text, where it appeared and the furthest stage reached. A menu that rejects metadata, a loader that rejects execution and an application that crashes after startup are different cases. The same screen description can hide different causes unless timing and artifact identity are recorded.

Folder names and installation structure

The laboratory publisher accepts an application folder of 1–32 ASCII bytes, beginning with a letter or digit and continuing with letters, digits, underscore or hyphen. It also rejects reserved Windows names such as CON, AUX, NUL, PRN and the listed COM/LPT forms. These are deliberate publishing and cross-platform handling policies.

The 32-byte policy is not a verified Sony OFW directory-name limit. No universal limit is established by the primary evidence used here. Test reports may document specific successful or failed names, but must retain their firmware, model, medium, encoding and application conditions. An emulator experiment has its own scope.

Use SampleApp as the baseline folder. If investigating a name-related failure, hold the binary and all other layout conditions constant, change one name at a time on a working copy and preserve the observed result. Avoid mixing a new build, new card, different path and longer name in the same experiment.

The archive's PSP directory should merge at the intended storage root. A user should not have to infer whether to copy the ZIP, an outer release folder or a nested PSP tree. Explain the expected resulting path plainly in the README, while keeping internal-flash and removable-media cases explicit where they differ.

Metadata and stale generated files

A hand-edited title does not repair an incorrectly typed SFO entry. Regenerate metadata from the identified recipe and validate its fields. Check MEMSIZE and PSP_SYSTEM_VER every time, along with title, category and bootable value. The declared firmware field should not be rewritten merely to make an error message disappear without investigating the executable's real dependencies.

Build systems can reuse old generated files. If a flag changes but the SFO target has no dependency that forces regeneration, a successful make can still package stale metadata. Compare the generated SFO and inspect a clean build before assuming that the source Makefile reflects the release.

Likewise, preserve the distinction between the unsigned PBP, signed PBP and installed EBOOT.PBP. A filename reused through several stages makes it easy to copy the wrong file. Stage the identified signed artifact into a new release tree, then verify its hash again.

Unsupported imports and privilege confusion

An unresolved or unexpected dependency should lead back to the import inventory and link map. Do not relabel an UNKNOWN NID as safe because its symbol looks familiar, and do not declare every sceKernel function privileged. The lightweight-mutex correction in Chapter 4 illustrates why names need exact source evidence.

Changing the module attribute or enabling kernel libraries is not an ordinary repair for a user-mode baseline. It changes the program's privilege and dependency contract. The laboratory rejects that change rather than pretending the original stock-OFW objective still describes the new artifact.

Optional service initialization should be checked at each stage. A PRX file that exists, a library that links on the host and an API declaration in a header are three different kinds of evidence. None alone establishes successful loading and operation on the target.

HOME callback, memory and apparent hangs

A visible HOME dialog is not proof of a complete exit path. The SDK documents registration of an exit callback and callback-aware thread waiting [S12]. Review callback creation, registration results, whether the servicing thread is scheduled and whether shutdown can wait on another blocked operation.

Keep exit handling appropriate to the application's complexity. A tiny example can have a simple path, while a streaming application may need to stop workers and release resources without deadlocking. Verify actual device behavior before claiming that the exit procedure is reliable.

Insufficient memory often reveals itself during the first substantial resource allocation, after a successful initial screen. Check the allocation result, requested capacity, largest free block and preceding service allocations. A successful startup does not demonstrate that the largest media workload fits.

When a program appears frozen, collect enough information to separate a busy loop, blocked thread, unserviced callback, failed allocation and rendering stall. Increasing CPU frequency, signing again or changing firmware metadata without a diagnosis can obscure the original failure.

Chapter 10 — Community Release Checklist

Construct a new staging tree

A release is a deliberate snapshot, not an accumulation of whatever happens to remain in a build directory. Create an empty staging tree and copy only the identified installation artifact, required documentation and selected evidence into it. Keep temporary mutations, private inputs and unrelated project files outside.

A small installation archive can use this layout. The exact companion manifest determines the files supplied with this guide.

```
README.md
LICENSE.txt
VALIDATION.md
PSP/
  GAME/
    SampleApp/
      EBOOT.PBP
SHA256SUMS.txt
```

The broader source-and-tests companion also needs the complete guide tools, SampleApp source and Makefile, tests, dependency locks, source pins, reproducibility instructions and evidence reports. Include compatibility conditions and known limitations as ordinary release documentation. A source ZIP containing only a binary and a build command is not the laboratory described here.

Use clear ownership for licenses. The guide's own source license does not automatically cover third-party tools, fonts, cryptographic code or embedded material. Preserve applicable notices and component provenance. The signing-tool research records unresolved clearance for the complete legacy bundle; do not replace that uncertainty with an invented uniform license.

Create a fresh archive

Updating an existing ZIP is unsafe for this workflow because removed staging files can survive as old archive members. Packaging only PSP/GAME is also incomplete when README, license and validation reports live at the staging root. Both failures can produce an archive that installs while omitting the evidence and instructions promised by the release.

The laboratory archive writer refuses an existing output path. It recursively includes the current staging files, checks required root documentation, rejects symlinks and case-colliding paths, and writes a fresh manifest. The output lives outside the staging tree so it cannot be recursively included in itself.

If the chosen output already exists, preserve it and select a new deliberate publication destination. Do not interpret the refusal as an instruction to overwrite a user's archive. Rebuild from the intended staging snapshot, then compare the new result with its own manifest.

Safe member names exclude absolute paths, parent traversal, backslashes, drive-style colons and control characters. Explicit case-collision checks make the archive more predictable when extracted on different host filesystems. These are publisher safety policies; they are not claims about every valid name Sony's loader can interpret.

Verify the exact member set and content

After creation, reopen the ZIP and verify its member set, uniqueness, integrity and hashes. Checking only that EBOOT.PBP is present misses stale extras and omitted root files. The verifier must require that every payload member is listed in the manifest and that every listed member exists with the expected bytes.

SHA256SUMS cannot contain its own final hash without a circular dependency. The internal manifest covers the other members; the archive itself is hashed in the external publication manifest. When an explicit individual hash for the internal manifest is required, record it in an external inventory after

archive creation. Keep this relationship documented instead of claiming a mathematically impossible self-hashing file.

A ZIP CRC and SHA-256 serve different purposes. CRC integrity does not replace the expected content manifest. SHA-256 identity does not prove licensing, behavior or compatibility. Keep the archive verification report and its precise scope with the release.

The source-and-tests ZIP attached to the PDF and the separately distributed ZIP should be byte-identical. Freeze the companion before embedding it. Complete the PDF afterward, then generate the external hashes for final deliverables. Do not put the final PDF hash inside a companion that the PDF itself embeds; that creates another circular construction.

Write a README a reader can act on

Lead with what SampleApp does and how to install its identified artifact. Explain the expected storage tree, how to launch it, how to leave it and which files are documentation. A reader should not need the original chat to understand the release.

Then state the evidence boundary: host validation results, emulator observations if performed, and physical model/firmware cases still NOT TESTED. Include known limitations, dependency requirements, storage or audio-route distinctions and any operation that is intentionally outside the example's scope.

For developers, identify the source commit, environment pin, full build commands, required retrieval steps and how to regenerate reports. Explain which upstream material is referenced rather than bundled. Keep tool revisions intact even though the document title and final filenames carry no artificial edition labels.

A useful issue report template asks for the EBOOT hash, full model, firmware and CFW/OFW state, storage medium, installation path, error stage and exact message. Ask for the smallest reproducible scenario. Avoid collecting unrelated private files when a short log or hash is sufficient.

A full publication workflow

Perform the following sequence for each release candidate:

1. Freeze the intended source snapshot and record its identity and dirty state.
2. Rebuild in the locked environment and preserve all relevant intermediate artifacts.
3. Run structural, typed-SFO, module and import checks against the actual unsigned PBP.
4. Sign through the isolated wrapper into a new output and retain the signer identity.
5. Cross-validate cryptography, expected relocation transformation and unchanged resources.
6. Run host tests and documented integration scenarios; preserve failures and reruns.
7. Build a new staging tree from the checked signed artifact and current documentation.
8. Create a new archive, reopen it and verify all expected members and content hashes.
9. Generate the guide's result tables from the same artifact reports.
10. Freeze and embed the companion, finish the PDF and verify the attachment identity.
11. Inspect every rendered page, links, listings, tables and source references.
12. Obtain independent technical review and record unresolved limitations.
13. Hash the final PDF, companion and audit deliverables in the external manifest.

These steps produce a concrete release candidate for public posting. They do not authorize posting to an account or communicating with a community on the reader's behalf. The release text should make the evidence easy to evaluate without overstating hardware support.

Compatibility matrix discipline

A matrix cell needs an exact case definition. “PSP-3000” alone is too broad if firmware, binary, medium and scenario are unspecified. Separate XMB recognition, entry, basic interaction, HOME exit, media behavior and suspend/resume when the release makes claims about them.

Do not fill blank cells with assumptions inherited from another family. Keep all unexecuted physical cases NOT TESTED, and leave performance NOT MEASURED. A documented FAIL is useful information for a community reader deciding whether to investigate or avoid a particular case.

A release can be valuable with those open rows. Its value comes from complete source, reproducible procedures, precise offline checks and honest boundaries. The next contributor can then add a specific observation without having to undo an unsupported universal compatibility claim.

Chapter 11 — Common PSP Homebrew Myths & Misconceptions

Signed EBOOT means guaranteed OFW launch

Signing changes a particular representation of executable content. It does not repair the application, supply missing exports, authorize kernel dependencies or prove that the target loader accepts every field. Require the full chain of structural, metadata, dependency, cryptographic and target-runtime evidence. Use “offline verification PASS; physical OFW NOT TESTED” when that is the actual result.

PPSSPP PASS means PSP hardware PASS

An emulator result belongs to the emulator and its configuration. It can expose real application defects and demonstrate modeled behavior, but it does not establish Sony OFW enforcement, physical memory conditions or device peripherals. Preserve emulator evidence and hardware evidence in separate rows even when both eventually succeed.

A 4 MiB signer buffer is the application's RAM limit

The legacy 4 MiB buffer is host memory used while reading a raw executable. It motivates a pre-invocation input bound. The running PSP program has a different budget involving static storage, heap, stacks, graphics and services. Neither a 5.58 MB encrypted output nor a compressed package size measures that budget.

Every sceKernel function requires Kernel Mode

The name is not an access-control specification. Public user stubs include sceKernel-prefixed functions, and a misleading annotation can confuse a mutex import with interrupt control. Use the exact library/NID mapping and source class, then keep target export and privilege acceptance separate from that mapping.

A matching SHA-256 proves the program is correct

A hash ties a name or report to exact bytes. It does not show that those bytes meet a format, implement the intended behavior or carry a particular license. Hash the artifact so that validation and runtime evidence remain attached to the same bytes; do not substitute identity for the checks themselves.

Successful PRX decryption proves Sony's loader will accept it

Decryption and CMAC checks establish properties of the selected cryptographic layout. A complete plaintext comparison establishes continuity with the expected transformed input. Sony's loader can still have other acceptance conditions, and the application can still have unsupported dependencies or runtime defects. The independent verifier is valuable precisely because its scope is explicit.

CFW compatibility establishes stock OFW compatibility

A modified environment can change the rules under which software is admitted or executed. Preserve the firmware state in every result and test stock firmware separately when that is the claim. If the device state cannot be established, mark it UNKNOWN rather than omitting the uncertainty.

A successful command completes the release

An output file can be stale, incomplete or associated with the wrong input. A signer exit code cannot substitute for cross-validation, and a zip command cannot substitute for member verification. Keep original inputs, use fresh destinations and connect each report to exact artifacts. Publication readiness is the result of the complete workflow, with open device claims still visible.

Primary Sources and Reference Identities

The companion research registers retain detailed source qualifications. The references below provide the identities used in this handbook. Hardware manufacturer claims and public SDK declarations have different scopes; no reference in this register is a substitute for the guide's actual test matrix.

ID	Primary source and scope
H1	Sony PSP-1000 launch specifications, 2004-10-27: physical RAM, embedded DRAM and interfaces.
H2	Sony PSP-2000 announcement, 2007-07-12: physical RAM and family features.
H3	Sony PSP-3000 announcement, 2008-09-02: physical RAM, microphone and interfaces.
H4	Sony PSP Go announcement, 2009-06-03: main RAM, internal flash, M2 and UMD limitation.
H5	Sony PSP-E1000 announcement, page dated 2011-08-17: main RAM, absent Wi-Fi and mono speaker.
H6	Sony PSP-E1000 English manual: System Information, specifications and media. Historical service descriptions do not establish current availability.
H7	PSPSDK GE header: VRAM-query and size-selection documentation.
H8	PSPSDK user sysmem header: allocation, free-memory and version interfaces.
H9	PSPSDK module-info header: inner metadata and attributes.
H10	PSPSDK import-stub macros: library and NID representation.
H11	PSPSDK module-manager header: lifecycle and loading restrictions.
H12	Unified utility modules and older AV modules: loader namespaces and documented dependencies.
H13	PSPSDK controller header: HOLD bit declaration.
S1	Legacy ebootsigner entry and KIRK implementation: selected transformation and crypto family.
S2	Legacy crypter: fixed layout, reader and late size comparison.
S3	SDK PrxEncrypter and templates: mode and selector facts.
S4	sign_np CLI, ELF implementation and AMCTRL: distinct ELF and NPUMDIMG paths.
S5	PSPPack CLI and signing implementation: minimal PBP and compression behavior.
S6	PrxEncrypter-Mod: name disambiguation and distinct implementation.
S7	Pinned SDK build.mak: compiler, PRX, import-fixup, SFO and packaging rules.
S8	Pinned mksfoex: SFO fields, types, defaults and generation.
S9	Pinned pack-pbp: PBP header and section order.
S10	Pinned legacy relocation pass: type-7 transformation.
S11	Pinned Kernel_Library user stub: exact mutex and interrupt NID mappings.
S12	Pinned thread header and exit header: thread attributes and callback interfaces.
S13	Pinned power header: clock-setting interface and parameter relationships.

All source observations remain tied to those revisions. Moving branches, similarly named forks and later tools require a new review. The final artifact and execution identities are supplied by the laboratory reports, not by this reference list.